Рекурсивні функції систем числення

Звіт • Джамалов Олександр Олегович • PI-11

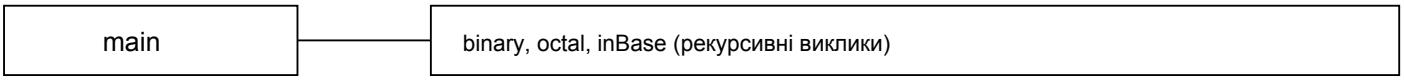
Для всіх варіантів визначити та вивести глибину та рівень рекурсії.

Варіант 7.

Визначити рекурсивні функції зображення натурального числа у двійковій, вісімковій системах числення та у системі числення з основою $N > 1$ та глибину рекурсії. Числа у десятковій системі числення вводити з клавіатури.

Мета: перевести натуральне число в основи 2, 8 і N > 1.
Рекурсія обробляє n / N; остача n % N додається на підйомі.
Базовий випадок: n < N. Перший рівень = 1; depth = max(level).
Глибина дорівнює кількості цифр результату в обраній системі.
Цифри 10..35 позначено A..Z, цифри ≥ 36 - як [значення].
Вхід: натуральні n та N у межах типу long long, N > 1.

Структура: main → binary, octal, inBase; кожна викликає себе



Git-репозиторій: _____

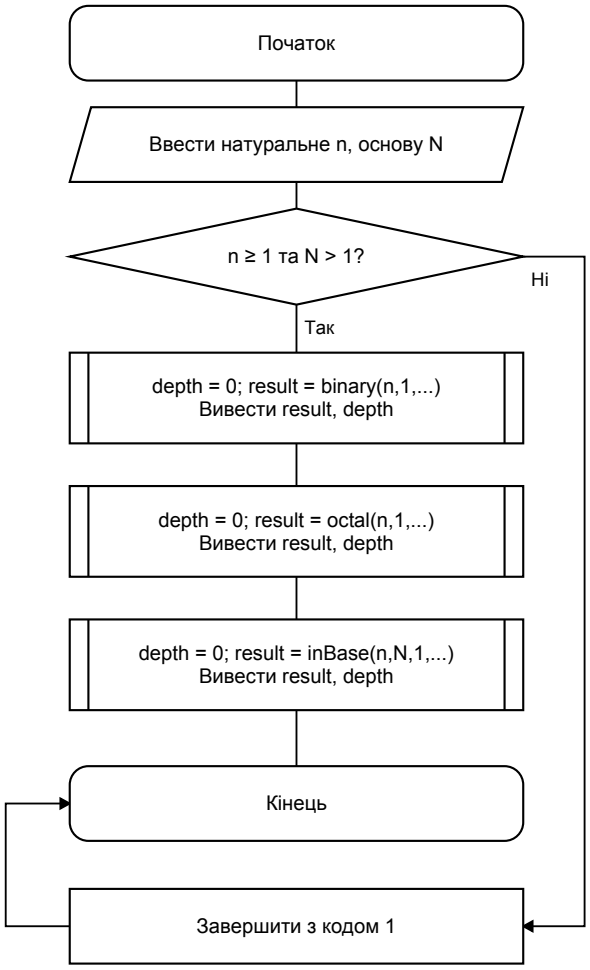
Приклад: n = 13, N = 16

```
Natural number, base (>1): Binary:
level=1 n=13
level=2 n=6
level=3 n=3
level=4 n=1
result=1101 depth=4
Octal:
level=1 n=13
level=2 n=1
result=15 depth=2
Base 16:
level=1 n=13
result=D depth=1
```

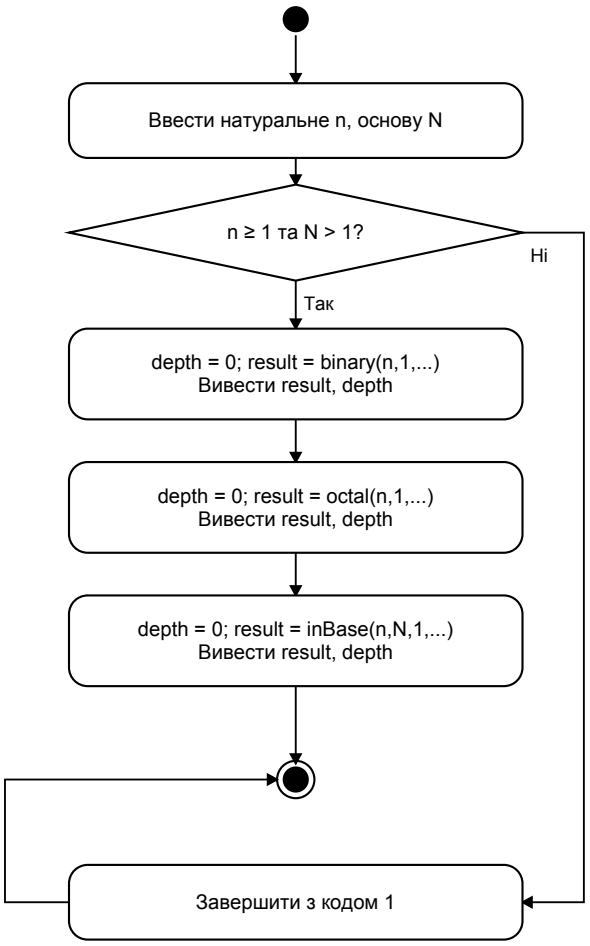
Алгоритм основної програми

main()

Блок-схема



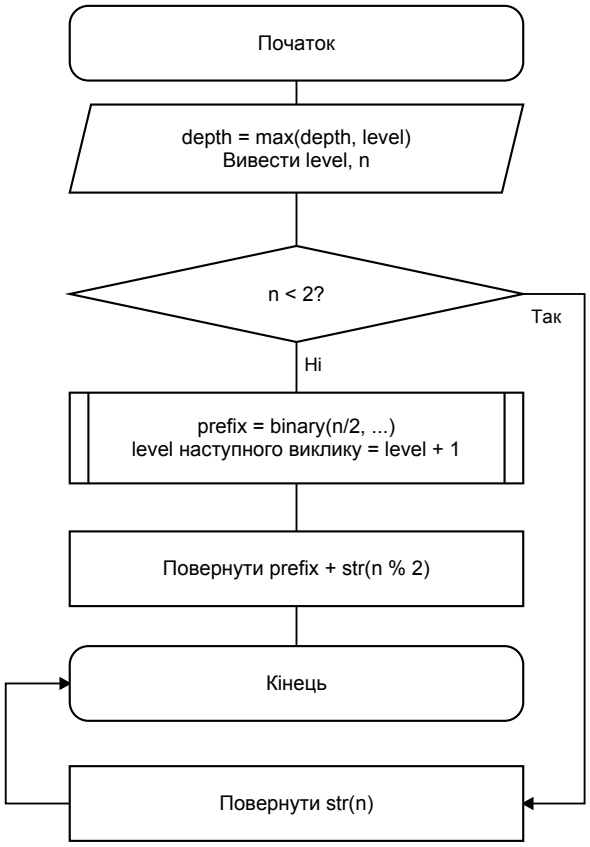
UML-діаграма діяльності



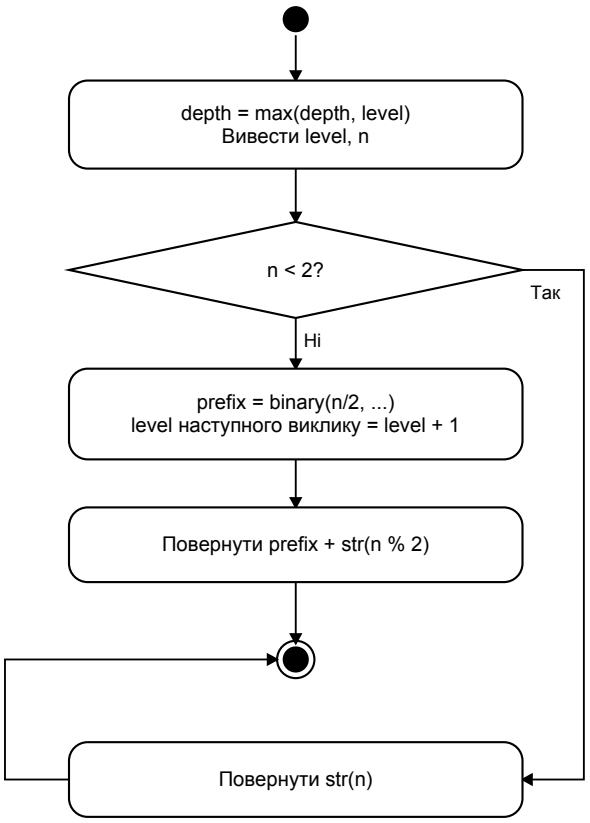
Двійкова та вісімкова системи

binary(n, level, depth, trace)

Блок-схема

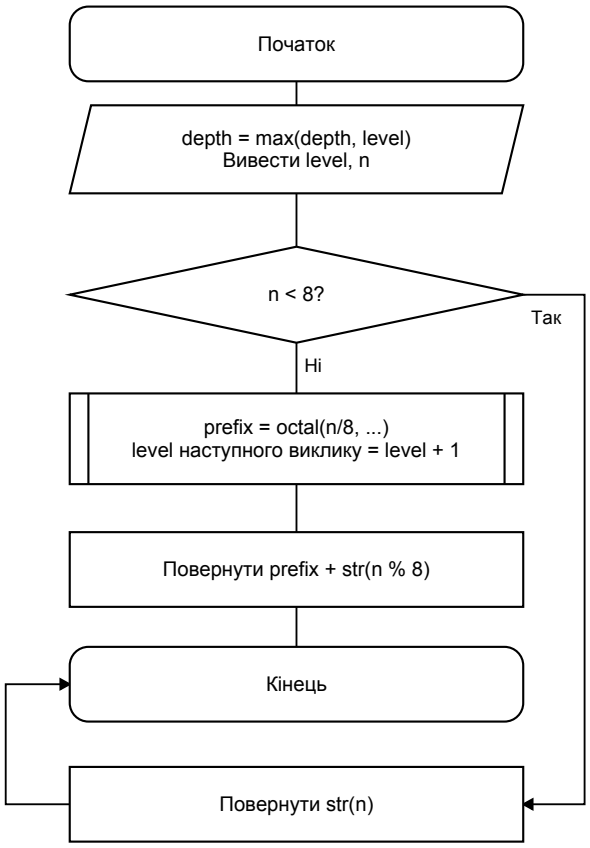


UML-діаграма діяльності

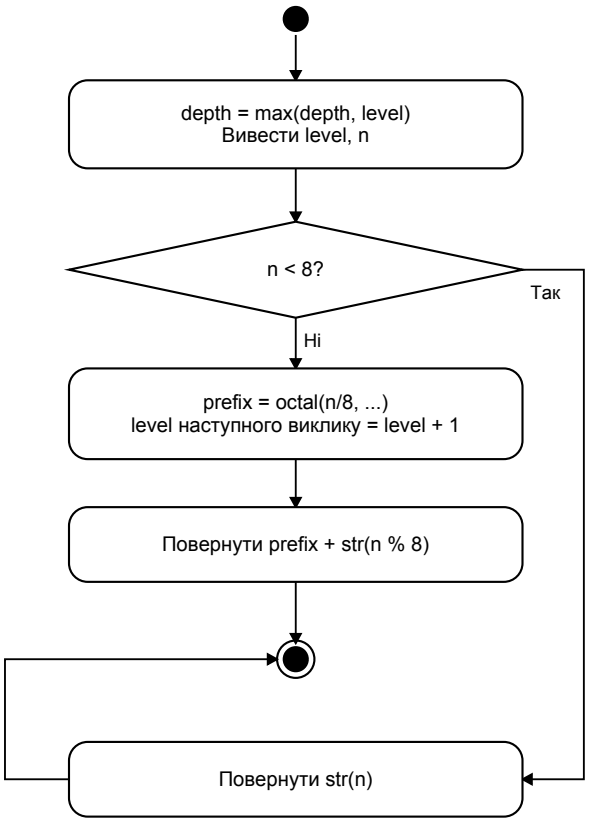


octal(n, level, depth, trace)

Блок-схема



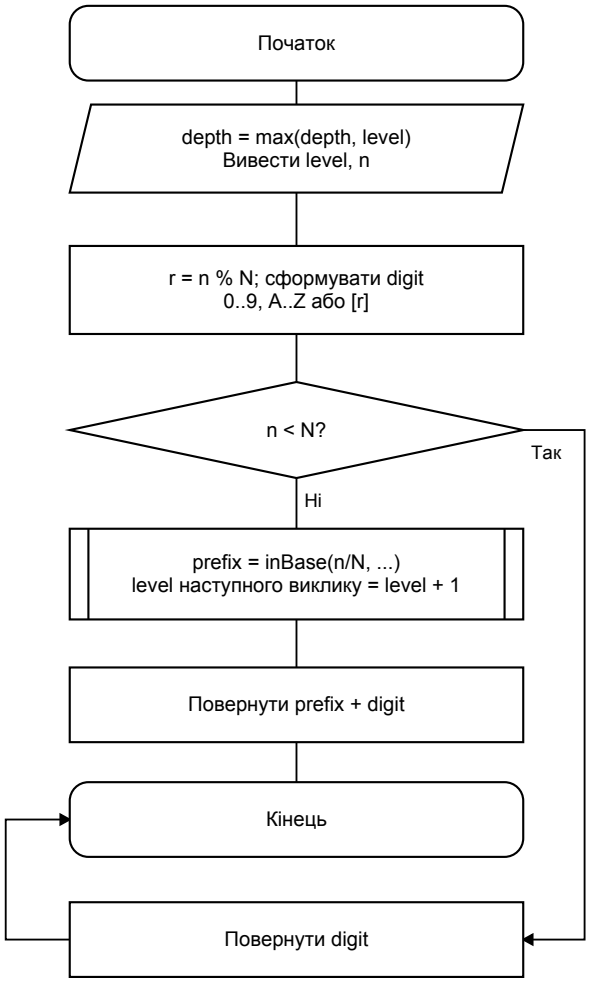
UML-діаграма діяльності



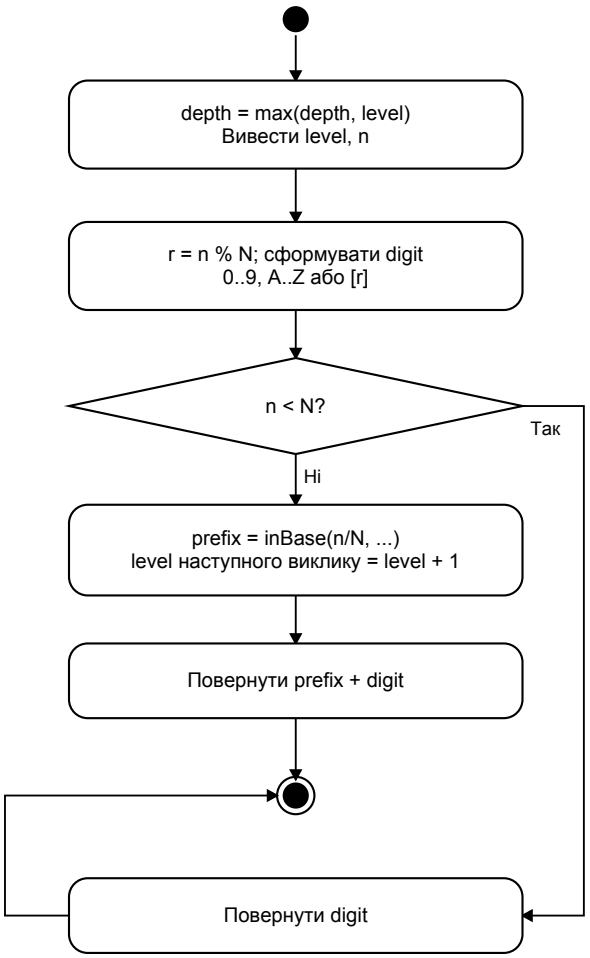
Довільна основа N > 1

inBase(n, N, level, depth, trace)

Блок-схема



UML-діаграма діяльності



Фактичний вивід unit-тесту

```
PASS n=1 base=2
PASS n=13 base=2
PASS n=64 base=8
PASS n=255 base=16
PASS n=79 base=40
PASS n=7 base=100
5.5: 6/6 passed
```

Перевірено результат, глибину та кількість виведених рівнів.

Висновок: три рекурсивні функції працюють; усі 6 тестів пройдено.

Текст програми та unit-тести

lab_5_5.cpp

```
#include <algorithm>
#include <iostream>
#include <string>
using namespace std;

string binary(long long n, int level, int& depth, ostream& trace)
{
    depth = max(depth, level);
    trace << "level=" << level << " n=" << n << '\n';
    if (n < 2)
        return to_string(n);
    return binary(n / 2, level + 1, depth, trace)
           + to_string(n % 2);
}

string octal(long long n, int level, int& depth, ostream& trace)
{
    depth = max(depth, level);
    trace << "level=" << level << " n=" << n << '\n';
    if (n < 8)
        return to_string(n);
    return octal(n / 8, level + 1, depth, trace)
           + to_string(n % 8);
}

// Digits >= 36 use [value], e.g. 79 in base 40 is 1[39].
string inBase(long long n, long long base, int level,
              int& depth, ostream& trace)
{
    depth = max(depth, level);
    trace << "level=" << level << " n=" << n << '\n';
    const string digits = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    const long long r = n % base;
    const string digit = r < 36 ? string(1, digits[r])
                                : "[" + to_string(r) + "]";
    if (n < base)
        return digit;
    return inBase(n / base, base, level + 1, depth, trace)
           + digit;
}

#ifndef UNIT_TEST
int main()
{
    long long n, base;
    cout << "Natural number, base (>1): ";
    if (!(cin >> n >> base) || n < 1 || base < 2)
        return 1;
    int depth = 0;
    cout << "Binary:\n";
    string result = binary(n, 1, depth, cout);
    cout << "result=" << result << " depth=" << depth << '\n';
    depth = 0;
    cout << "Octal:\n";
    result = octal(n, 1, depth, cout);
    cout << "result=" << result << " depth=" << depth << '\n';
    depth = 0;
    cout << "Base " << base << ":\n";
    result = inBase(n, base, 1, depth, cout);
```

Текст програми та unit-тести

```
    cout << "result=" << result << " depth=" << depth << '\n';
    return 0;
}
#endif
```

test_5_5.cpp

```
#define UNIT_TEST
#include "lab_5_5.cpp"
#include <sstream>

int main()
{
    struct Case { long long n, base; string result; int depth; };
    const Case cases[] = {{1, 2, "1", 1}, {13, 2, "1101", 4},
        {64, 8, "100", 3}, {255, 16, "FF", 2},
        {79, 40, "1[39]", 2}, {7, 100, "7", 1}};
    int failed = 0;
    for (const auto& v : cases)
    {
        int depth = 0;
        ostringstream trace;
        const string actual = inBase(v.n, v.base, 1, depth, trace);
        bool ok = actual == v.result && depth == v.depth;
        const string log = trace.str();
        ok = ok && count(log.begin(), log.end(), '\n') == depth;
        if (v.base == 2)
        {
            depth = 0;
            ok = ok && binary(v.n, 1, depth, trace) == v.result
                    && depth == v.depth;
        }
        if (v.base == 8)
        {
            depth = 0;
            ok = ok && octal(v.n, 1, depth, trace) == v.result
                    && depth == v.depth;
        }
        cout << (ok ? "PASS" : "FAIL") << " n=" << v.n
             << " base=" << v.base << '\n';
        if (!ok) ++failed;
    }
    cout << "5.5: " << 6 - failed << "/6 passed\n";
    return failed ? 1 : 0;
}
```